

Task 1

Write a C program where a parent process creates multiple child processes and synchronizes their completion using wait() and waitpid(). Compare the behavior of both functions

```
shivam@MSI123:~$ nano ptask1.c
shivam@MSI123:~$ |
```

```
GNU nano 7.2 Ptasi
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    int i, status;
    pid_t pid;
    printf("Parent Process ID: %d\n", getpid());
    // Create 3 child processes
    for (i = 1; i <= 3; i++)
    {
        pid = fork();
        if (pid < 0)
        {
            perror("fork failed");
            exit(1);
        }
        if (pid == 0)
        {
            printf("Child %d: PID = %d, Parent PID = %d\n",
                   i, getpid(), getppid());
            sleep(i);
            printf("Child %d completed.\n", i);
            exit(i);
        }
    }
    // Parent waits for all children
    for (i = 1; i <= 3; i++)
```

```

// Parent waits for all children
for (i = 1; i <= 3; i++)
{
    pid = wait(&status);
    if (WIFEXITED(status))
    {
        printf("Parent: Child with PID %d finished with exit status %d\n",
               pid, WEXITSTATUS(status));
    }
}
printf("All child processes completed.\n");
return 0;
}

```

Output:

```

shivam@MSI123:~$ nano ptask1.c
shivam@MSI123:~$ gcc ptask1.c -o ptask1
shivam@MSI123:~$ ./ptask1
Parent Process ID: 561
Child 1: PID = 562, Parent PID = 561
Child 2: PID = 563, Parent PID = 561
Child 3: PID = 564, Parent PID = 561
Child 1 completed.
Parent: Child with PID 562 finished with exit status 1
Child 2 completed.
Parent: Child with PID 563 finished with exit status 2
Child 3 completed.
Parent: Child with PID 564 finished with exit status 3
All child processes completed.

```

Task 2

Create a scenario where a child process becomes a zombie process. Investigate the process table and then modify the program to eliminate zombie processes using proper synchronization techniques.

```

shivam@MSI123:~$ nano ptask2.c
shivam@MSI123:~$ |

```

```

GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
    int i, status;
    pid_t pid[3];
    printf("Parent Process ID: %d\n", getpid());
    // Create 3 child processes
    for (i = 0; i < 3; i++)
    {
        pid[i] = fork();
        if (pid[i] < 0)
        {
            perror("fork failed");
            exit(1);
        }
        if (pid[i] == 0)
        {
            printf("Child %d: PID = %d, Parent PID = %d\n",
                    i + 1, getpid(), getppid());
            sleep(3 - i);
            printf("Child %d completed.\n", i + 1);
            exit(i + 1);
        }
    }
}

```

```

}
// Parent waits for specific children
for (i = 0; i < 3; i++)
{
    waitpid(pid[i], &status, 0);
    if (WIFEXITED(status))
    {
        printf("Parent: Child %d with PID %d finished with status %d\n",
                i + 1, pid[i], WEXITSTATUS(status));
    }
}
printf("All child processes completed.\n");
return 0;
}

```

OUTPUT:

```
shivam@MSI123:~$ gcc ptask2.c -o ptask2
shivam@MSI123:~$ ./ptask2
Parent Process ID: 627
Child 1: PID = 628, Parent PID = 627
Child 2: PID = 629, Parent PID = 627
Child 3: PID = 630, Parent PID = 627
Child 3 completed.
Child 2 completed.
Child 1 completed.
Parent: Child 1 with PID 628 finished with status 1
Parent: Child 2 with PID 629 finished with status 2
Parent: Child 3 with PID 630 finished with status 3
All child processes completed.
shivam@MSI123:~$ |
```